Programming Challenges

Questions and Solutions

Python Programs

Part A: Questions

Section 1: Input and Calculation Programs

Question 1: Ingredient Adjuster

A cookie recipe calls for 1.5 cups of sugar, 1 cup of butter, and 2.75 cups of flour. The recipe produces 48 cookies with this amount of ingredients.

Write a program that asks the user how many cookies he or she wants to make, then displays the number of cups of each ingredient needed for the specified number of cookies.

Question 2: Stock Transaction Program

Last month, Joe purchased stock in Acme Software, Inc. The purchase details were:

- Number of shares purchased: 2,000
- Price paid per share: $40.00
- Commission paid to the stockbroker: 3 percent of the amount paid for the stock

Two weeks later, Joe sold the stock. The sale details were:

- Number of shares sold: 2,000
- Sale price per share: $42.75
- Commission paid to the stockbroker: 3 percent of the amount received for the stock

Write a program that displays the following:

- The amount of money Joe paid for the stock
- The commission Joe paid his broker when he bought the stock
- The amount for which Joe sold the stock
- The commission Joe paid his broker when he sold the stock
- The amount of money Joe had left after selling the stock and paying the broker both times. If positive, Joe made a profit. If negative, Joe lost money.

Question 3: Planting Grapevines

A vineyard owner is planting new rows of grapevines and needs to know how many vines to plant in each row. After measuring a row, she uses the following formula:

$$V = (R - 2E) / S$$

Where:

- V is the number of grapevines that will fit in the row.
- R is the length of the row, in feet.
- E is the space, in feet, used by an end-post assembly.
- S is the space between vines, in feet.

Write a program that asks the user to input the row length, the space used by an end-post assembly, and the space between the vines, all in feet. The program should then calculate and display the number of grapevines that will fit in the row.

Section 2: Conditional Statement Challenges

Question 4: Hot Dog Cookout Calculator

Hot dogs come in packages of 10, and hot dog buns come in packages of 8. Write a program that calculates the number of packages of hot dogs and buns needed for a cookout, with the minimum amount of leftovers.

The program should ask for the number of people attending and the number of hot dogs each person will be given. It should display:

- The minimum number of packages of hot dogs required
- The minimum number of packages of hot dog buns required
- The number of hot dogs left over
- The number of hot dog buns left over

Question 5: Roulette Wheel Colors

On a roulette wheel, the pockets are numbered 0 to 36. The colors are as follows:

- Pocket 0 is green.
- Pockets 1 through 10: odd-numbered pockets are red, even-numbered pockets are black.
- Pockets 11 through 18: odd-numbered pockets are black, even-numbered pockets are red.
- Pockets 19 through 28: odd-numbered pockets are red, even-numbered pockets are black.
- Pockets 29 through 36: odd-numbered pockets are black, even-numbered pockets are red.

Write a program that asks the user to enter a pocket number and displays whether the pocket is green, red, or black. If the number is outside the range 0 through 36, the program should display an error message.

Question 6: Time Calculator

Write a program that asks the user to enter a number of seconds and works as follows:

- There are 60 seconds in a minute. If the number of seconds is 60 or more, convert it to minutes and seconds.
- There are 3,600 seconds in an hour. If the number of seconds is 3,600 or more, convert it to hours, minutes, and seconds.
- There are 86,400 seconds in a day. If the number of seconds is 86,400 or more, convert it to days, hours, minutes, and seconds.

Part B: Solutions

Solution 1: Ingredient Adjuster

Program (Python)

```
cookies = float(input("How many cookies do you want to make? "))

BASE_COOKIES = 48
factor = cookies / BASE_COOKIES

print(f"Sugar: {1.5 * factor:.2f} cups")
print(f"Butter: {1.0 * factor:.2f} cups")
print(f"Flour: {2.75 * factor:.2f} cups")
```

Sample run

```
How many cookies do you want to make? 96
Sugar: 3.00 cups
Butter: 2.00 cups
Flour: 5.50 cups
```

Explanation

The program finds a scale factor by dividing the requested number of cookies by the 48 cookies the recipe produces, then multiplies each ingredient by that factor.

Solution 2: Stock Transaction Program

Program (Python)

```
SHARES = 2000
BUY_PRICE = 40.00
SELL_PRICE = 42.75
COMMISSION_RATE = 0.03

# Purchase
amount_paid = SHARES * BUY_PRICE
buy_commission = amount_paid * COMMISSION_RATE

# Sale
amount_received = SHARES * SELL_PRICE
sell_commission = amount_received * COMMISSION_RATE

# Money left after both transactions and both commissions
left_over = (amount_received - sell_commission) - (amount_paid + buy_commission)

print(f"Amount paid for the stock: ${amount_paid:,.2f}")
print(f"Commission paid when buying: ${buy_commission:,.2f}")
print(f"Amount received from the sale: ${amount_received:,.2f}")
print(f"Commission paid when selling: ${sell_commission:,.2f}")
print(f"Money left after the sale and both commissions: ${left_over:,.2f}")

if left_over > 0:
    print("Joe made a profit.")
elif left_over < 0:
    print("Joe lost money.")
else:
    print("Joe broke even.")
```

Sample run

```
Amount paid for the stock: $80,000.00
Commission paid when buying: $2,400.00
Amount received from the sale: $85,500.00
Commission paid when selling: $2,565.00
Money left after the sale and both commissions: $535.00
Joe made a profit.
```

Explanation

Joe paid $82,400.00 in total to buy the stock and received $82,935.00 after selling and paying the second commission, leaving a profit of $535.00.

Solution 3: Planting Grapevines

Program (Python)

```
row_length = float(input("Enter the length of the row, in feet: "))
end_post_space = float(input("Enter the space used by an end-post assembly, in
feet: "))
vine_space = float(input("Enter the space between the vines, in feet: "))

# Formula: V = (R - 2E) / S
# Partial vines cannot be planted, so the result is truncated to a whole number.
vines = int((row_length - 2 * end_post_space) / vine_space)

print(f"Number of grapevines that will fit in the row: {vines}")
```

Sample run

```
Enter the length of the row, in feet: 100
Enter the space used by an end-post assembly, in feet: 3
Enter the space between the vines, in feet: 8
Number of grapevines that will fit in the row: 11
```

Explanation

Using the formula, (100 − 2 × 3) / 8 = 11.75. Only whole vines can be planted, so the program displays 11.

Solution 4: Hot Dog Cookout Calculator

Program (Python)

```
import math

HOT_DOGS_PER_PACKAGE = 10
BUNS_PER_PACKAGE = 8

people = int(input("Enter the number of people attending: "))
hot_dogs_each = int(input("Enter the number of hot dogs each person will be given:
"))

hot_dogs_needed = people * hot_dogs_each
buns_needed = hot_dogs_needed  # one bun for each hot dog

hot_dog_packages = math.ceil(hot_dogs_needed / HOT_DOGS_PER_PACKAGE)
bun_packages = math.ceil(buns_needed / BUNS_PER_PACKAGE)

hot_dogs_left_over = hot_dog_packages * HOT_DOGS_PER_PACKAGE - hot_dogs_needed
```

```
buns_left_over = bun_packages * BUNS_PER_PACKAGE - buns_needed

print(f"Packages of hot dogs required: {hot_dog_packages}")
print(f"Packages of hot dog buns required: {bun_packages}")
print(f"Hot dogs left over: {hot_dogs_left_over}")
print(f"Hot dog buns left over: {buns_left_over}")
```

Sample run

```
Enter the number of people attending: 25
Enter the number of hot dogs each person will be given: 3
Packages of hot dogs required: 8
Packages of hot dog buns required: 10
Hot dogs left over: 5
Hot dog buns left over: 5
```

Explanation

The 25 guests need 75 hot dogs and 75 buns. Eight packages of hot dogs give 80 (5 left over), and ten packages of buns give 80 (5 left over). Math.ceil rounds up to the smallest number of whole packages that covers the need.

2nd Method

```
people = int(input("Please enter the number of people attending this cookout: "))
hot_dogs_each = int(input("Please enter the number of hot dogs for each person:
"))

hot_dogs_needed = people * hot_dogs_each
buns_needed = hot_dogs_needed

hot_dog_packages = hot_dogs_needed // 10
if hot_dogs_needed % 10 > 0:
    hot_dog_packages += 1

bun_packages = buns_needed // 8
if buns_needed % 8 > 0:
    bun_packages += 1

hot_dogs_left_over = hot_dog_packages * 10 - hot_dogs_needed
buns_left_over = bun_packages * 8 - buns_needed

print("The minimum number of packages of hot dogs required:", hot_dog_packages)
print("The minimum number of packages of hot dog buns required:", bun_packages)
print("The number of hot dogs that will be left over:", hot_dogs_left_over)
print("The number of hot dog buns that will be left over:", buns_left_over)
```

Solution 5: Roulette Wheel Colors

Program (Python)

```
pocket = int(input("Enter a pocket number (0 through 36): "))

if pocket < 0 or pocket > 36:
    print("Error: the pocket number must be between 0 and 36.")
elif pocket == 0:
    print("Pocket 0 is green.")
elif (1 <= pocket <= 10) or (19 <= pocket <= 28):
    color = "red" if pocket % 2 == 1 else "black"
    print(f"Pocket {pocket} is {color}.")
else:  # pockets 11-18 or 29-36
    color = "black" if pocket % 2 == 1 else "red"
    print(f"Pocket {pocket} is {color}.")
```

Sample run

```
Enter a pocket number (0 through 36): 12
Pocket 12 is red.
```

Explanation

Pocket 0 is handled first. The two odd/even color patterns are then selected by checking which range the number falls in. Any number outside 0 through 36 produces the error message.

Solution 6: Time Calculator

Program (Python)

```
seconds = int(input("Enter a number of seconds: "))

if seconds >= 86400:
    days = seconds // 86400
    seconds = seconds % 86400
    hours = seconds // 3600
    seconds = seconds % 3600
    minutes = seconds // 60
    seconds = seconds % 60
    print(f"Days: {days}")
    print(f"Hours: {hours}")
    print(f"Minutes: {minutes}")
    print(f"Seconds: {seconds}")
elif seconds >= 3600:
    hours = seconds // 3600
    seconds = seconds % 3600
    minutes = seconds // 60
    seconds = seconds % 60
    print(f"Hours: {hours}")
    print(f"Minutes: {minutes}")
    print(f"Seconds: {seconds}")
elif seconds >= 60:
    minutes = seconds // 60
    seconds = seconds % 60
    print(f"Minutes: {minutes}")
    print(f"Seconds: {seconds}")
else:
    print(f"Seconds: {seconds}")
```

Sample run

```
Enter a number of seconds: 5000
```

```
Hours: 1
Minutes: 23
Seconds: 20
```

Explanation

The most specific range is checked first (days, then hours, then minutes). The // operator gives the whole units and % keeps the remainder for the next smaller unit.